

Лабораторная работа №1

Лимонов Дмитрий ИУ9-52Б

10 октября 2025 г.

Изначальная система переписывания

Рассматривается система переписывания строк (вариант 15), где алфавит

$$\Sigma = \{a, b, c\},$$

а множество правил задано следующим образом:

abca $\rightarrow$ aaaa,
bcab $\rightarrow$ bbbb,
cabc $\rightarrow$ cccc,
abc $\rightarrow$ aaa,
abc $\rightarrow$ bbb,
abc $\rightarrow$ ccc,
aabbcc $\rightarrow$ abcabc.

Завершимость

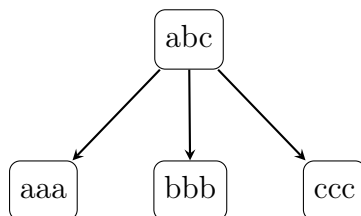
Рассмотрим правила переписывания. Все правила, кроме последнего, могут быть применены только в том случае, если в строке есть подстрока из 3 различных символов, идущих подряд (abc, bac, cab, ...), при этом все правила как раз убирают эти различные символы. Последнее же правило может быть применено в случае, если встречается подстрока bb. Так как среди предыдущих правил нет ни одного способа пополнить количество именно bb, то, в какой-то момент, переписывания остановятся

Конечность

Исходя из структуры правил можно заметить, что ни одно из них не уменьшает длины изначальной строки. Значит, как минимум, существуют классы эквивалентности по длине строки.

Конфлюэнтность

Изначальная система не является конфлюэнтной ни локально, ни глобально. Это можно увидеть на примере строки `abc`:



Дополним систему по алгоритму Кнута-Бендикса. Сначала введём лексикографический порядок в систему и переупорядочим правила:

`abca` $\rightarrow$ `aaaa`,
`bcab` $\rightarrow$ `bbbb`,
`cccc` $\rightarrow$ `cabc`,
`abc` $\rightarrow$ `aaa`,
`bbb` $\rightarrow$ `abc`,
`ccc` $\rightarrow$ `abc`,
`abcabc` $\rightarrow$ `aabbcc`.

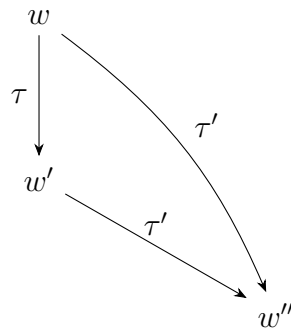
Далее, мною была написана программа по подбору новых правил. Исходный код программы можно найти [здесь](#). В результате выполнения программы стало ясно, что в системе при пополнении возникает бесконечное количество правил, поэтому возьмём приближённую эквивалентную систему (она будет полностью эквивалентна на строках длиной 7 и меньше.):

bcab $\rightarrow$ aaab	caaa $\rightarrow$ aaac	abc $\rightarrow$ aaa
bbb $\rightarrow$ aaa	ccc $\rightarrow$ aaa	aabbcc $\rightarrow$ aaaaaa
baaa $\rightarrow$ aaab	aaacc $\rightarrow$ aaaab	aaabab $\rightarrow$ aaaaac
aaabbc $\rightarrow$ aaabaa	aaacab $\rightarrow$ aaaaaa	aaacbc $\rightarrow$ aaacaa
aaaaabb $\rightarrow$ aaaaaac	aaabaac $\rightarrow$ aaaaaaa	aaabacc $\rightarrow$ aaabaab
aaabbba $\rightarrow$ aaaaaac	aaabbab $\rightarrow$ aaaaaaa	aaacacc $\rightarrow$ aaacaab
aaacbab $\rightarrow$ aaacaac	aaacbbc $\rightarrow$ aaacbaa	

Тестирование

Фаззинг

Исходный код программы для фаззинг-тестирования может быть найден в репозитории. Основная идея - сгенерировать случайное слово, далее привести его к нормальной форме в τ . После этого уже изначальное слово и полученную на предыдущем шаге нормальную форму перевести в общее слово, но уже в τ' . В моей программе, для простоты реализации, этим словом также будет нормальная форма, то есть:



Метаморфное тестирование

Исходный код программы для метаморфного тестирования также может быть найден в репозитории. В качестве инвариантов были выбраны две характеристики:

1. Длина слова
2. $(\Delta A - \Delta C) \bmod 3 = 0$, где ΔA и ΔC - разность числа букв **a** или **c** соответственно.